Buổi 11

Cây nhị phân tìm kiếm (BST)

Học phần: Cấu trúc dữ liệu và Giải thuật

Nội dung bài học

1. Khái niệm cây nhị phân tìm kiếm

1.1 Định nghĩa và tính chất
1.2 Cấu trúc dữ liệu và ý nghĩa của BST

2. Các thao tác trên BST

2.1 Tìm kiếm trên BST
2.2 Thêm một nút vào BST
2.3 Hủy nút trên BST – ba trường hợp
2.4 Hủy nút trên BST – cài đặt
2.5 Minh họa hủy nút gốc có hai con
2.6 Thêm nút không đệ quy và tạo cây
2.7 Hai phương án chọn phần tử thế mạng
2.8 Các thao tác khác trên BST

3. Độ phức tạp và trường hợp suy biến

3.1 Độ phức tạp theo chiều cao cây
3.2 So sánh BST với các cấu trúc đã học
3.3 BST với khóa là chuỗi và dữ liệu kèm theo

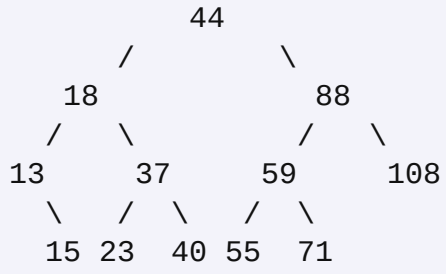
4. Tổng kết và thực hành

4.1 Bài thực hành buổi 11 – Từ điển Anh – Việt bằng BST
4.2 Câu hỏi và bài tập

Định nghĩa

Cây nhị phân tìm kiếm (Binary Search Tree – BST) là cây nhị phân mà tại **mọi nút**: Key của mọi nút trong **cây con trái nhỏ hơn** Key của nút đó, và Key của mọi nút trong **cây con phải lớn hơn** Key của nút đó. (Giả sử các Key khác nhau.)

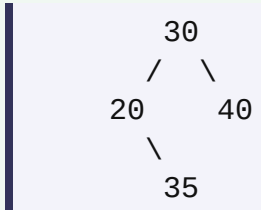
Ví dụ 11.1 – Cây B (buổi 10) là một BST



- Tại 44: cây trái {13, 15, 18, 23, 37, 40} < 44 < cây phải {55, 59, 71, 88, 108} ✓.
- Tại 37: 23 < 37 < 40 ✓.

- **Tính chất quan trọng:**
  - Duyệt **LNR** cho dãy **tăng dần**: 13 15 18 23 37 40 44 55 59 71 88 108.
  - Nút **nhỏ nhất** là nút **trái nhất** (đi trái đến hết): 13; nút **lớn nhất** là nút **phải nhất**: 108.
  - Mỗi cây con của BST cũng là một BST.

Ví dụ 11.2 – Không phải BST



35 nằm trong cây con **trái** của 30 nhưng 35 > 30 ⇒ **vi phạm** (dù tại nút 20: 20 < 35 vẫn đúng). Phải kiểm tra điều kiện với **mọi tổ tiên**, không chỉ với nút cha.

1.2 Cấu trúc dữ liệu và ý nghĩa của BST

```
typedef struct BST_Node
{  T Key;
   BST_Node *BST_Left;     // cây con trái
   BST_Node *BST_Right;    // cây con phải
} BST_OneNode;
typedef BST_OneNode *BST_Type;
BST_Type BST_Root;         // khởi tạo: BST_Root = NULL
```

- Cấu trúc nút **giống hệt** cây nhị phân thường – khác biệt nằm ở **ràng buộc thứ tự** của dữ liệu.

- **BST kết hợp ưu điểm của hai cấu trúc:**

Cấu trúc	Tìm kiếm	Thêm / hủy
Mảng có thứ tự	O(log N) – tìm nhị phân	O(N) – dời phần tử
DSLK	O(N) – tìm tuyến tính	O(1) khi biết vị trí
BST (cân bằng)	**O(log N)**	**O(log N)**

- Mỗi lần so sánh tại một nút, BST loại bỏ **một cây con** – giống tìm nhị phân loại một nửa dãy.
- **Ứng dụng:** từ điển, bảng ký hiệu của trình biên dịch, chỉ mục CSDL (B-cây), tập hợp và ánh xạ (`std::set`, `std::map`).

2.1 Tìm kiếm trên BST

- **Thuật toán:** bắt đầu từ gốc:
  - Cây rỗng ⇒ **không tìm thấy**.
  - X = Key ⇒ **tìm thấy**.
  - X < Key ⇒ tìm tiếp ở **cây con trái**.
  - X > Key ⇒ tìm tiếp ở **cây con phải**.

```
// Đệ quy
BST_Type BST_Search(BST_Type Root, T X)
{  if (Root == NULL || Root->Key == X)
      return Root;
   if (X < Root->Key)
      return BST_Search(Root->BST_Left, X);
   return BST_Search(Root->BST_Right, X);
}
// Không đệ quy
BST_Type BST_Search_NR(BST_Type Root, T X)
{  BST_Type p = Root;
   while (p != NULL && p->Key != X)
      p = (X < p->Key) ? p->BST_Left : p->BST_Right;
   return p;
}
```

Ví dụ 11.3 – Tìm trên cây B

X	Đường đi	Kết quả	Số phép so sánh Key
40	44 → 18 → 37 → **40**	Tìm thấy	4
59	44 → 88 → **59**	Tìm thấy	3
50	44 → 88 → 59 → 55 → (trái rỗng)	Không thấy	4
108	44 → 88 → **108**	Tìm thấy	3

- Tìm 50: 50 > 44 ⇒ phải; 50 < 88 ⇒ trái; 50 < 59 ⇒ trái; 50 < 55 ⇒ trái = NULL ⇒ dừng.

- **Phân tích:** số phép so sánh ≤ **chiều cao h** của cây ⇒ **O(h)**: cây cân bằng h ≈ $\log_2 N$; cây suy biến h = N.
- So với tìm trên cây nhị phân thường (buổi 10) phải xét **cả hai** cây con O(N).

2.2 Thêm một nút vào BST

- **Thuật toán:** tìm X như khi tìm kiếm; nếu đã có ⇒ không thêm (Key không trùng); nếu gặp vị trí **NULL** ⇒ tạo nút mới **tại chỗ đó** (nút mới luôn là **lá**).

```
int BST_Add_Node(BST_Type &Root, T NewData)
{  if (Root == NULL)                          // vị trí trống
   {  Root = BST_Create_Node(NewData);
      return (Root != NULL);
   }
   if (NewData == Root->Key)
      return 0;                               // đã tồn tại
   if (NewData < Root->Key)
      return BST_Add_Node(Root->BST_Left, NewData);
   return BST_Add_Node(Root->BST_Right, NewData);
}
```

- Tham số `BST_Type &Root`: khi Root là con trỏ BST_Left/BST_Right của nút cha, gán `Root = …` chính là **nối** nút mới vào cha.
- Thêm 37 lần nữa ⇒ trả về **0**.

Ví dụ 11.4 – Tạo cây B từ dãy 44, 18, 88, 13, 37, 59, 108, 15, 23, 40, 55, 71

Thêm	Đường đi	Vị trí
44	–	gốc
18	44 →	trái của 44
88	44 →	phải của 44
13	44 → 18 →	trái của 18
37	44 → 18 →	phải của 18
59, 108	44 → 88 →	trái, phải của 88
15	44 → 18 → 13 →	phải của 13
23, 40	44 → 18 → 37 →	trái, phải của 37
55, 71	44 → 88 → 59 →	trái, phải của 59

- **Thứ tự thêm quyết định hình dạng cây:** cùng tập dữ liệu, thứ tự khác ⇒ cây khác.

2.3 Hủy nút trên BST – ba trường hợp

- Tìm nút cần hủy (DelNode), xét số con của nó:

Trường hợp	Cách xử lý
1. Nút lá	Hủy, gán liên kết của cha về **NULL**
2. Có một con	Cho **cha** của DelNode trỏ thẳng tới **con duy nhất** ("nối tắt")
3. Có hai con	**Không** hủy trực tiếp. Tìm **phần tử thế mạng** Y, chép Y vào DelNode, rồi **hủy Y** (Y có tối đa 1 con ⇒ quay về TH 1, 2)

- **Phần tử thế mạng** (một trong hai):
  - Nút **trái nhất của cây con phải** (nhỏ nhất trong các nút lớn hơn DelNode).
  - Nút **phải nhất của cây con trái** (lớn nhất trong các nút nhỏ hơn DelNode).
- Thế mạng giữ được tính chất BST vì nó là phần tử **liền kề** DelNode trong dãy LNR tăng dần.

Ví dụ 11.5 – Trên cây B: hủy lần lượt 40, 13, 18, 44

```
Hủy 40 (lá):           37.Right = NULL
Hủy 13 (1 con 15):     18.Left → 15
          44
        /    \
      18      88
     /  \    /  \
   15   37  59  108
       /   /  \
      23  55  71
Hủy 18 (2 con): thế mạng = trái nhất cây phải = 23
          44
        /    \
      23      88
     /  \    /  \
   15   37  59  108
           /  \
          55  71
```

2.4 Hủy nút trên BST – cài đặt

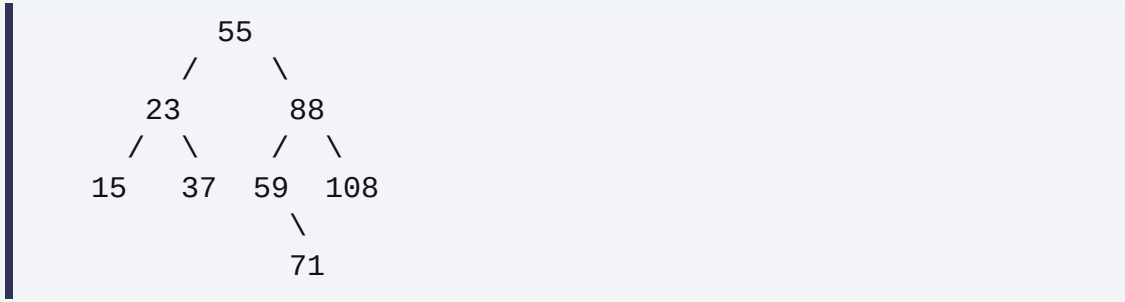
```
int BST_Delete_Node(BST_Type &Root, T X)
{  if (Root == NULL) return 0;          // không có X
   if (X < Root->Key) return BST_Delete_Node(Root->BST_Left, X);
   if (X > Root->Key) return BST_Delete_Node(Root->BST_Right, X);
   BST_Type DelNode = Root;             // Root->Key == X
   if (Root->BST_Left == NULL)          // TH 1, 2
      Root = Root->BST_Right;
   else if (Root->BST_Right == NULL)    // TH 2
      Root = Root->BST_Left;
   else                                 // TH 3: hai con
   {  BST_Type MinP = Root, MinNode = Root->BST_Right;
      while (MinNode->BST_Left != NULL) // trái nhất cây phải
      {  MinP = MinNode;  MinNode = MinNode->BST_Left;  }
      Root->Key = MinNode->Key;         // chép thế mạng
      if (MinP == Root) MinP->BST_Right = MinNode->BST_Right;
      else              MinP->BST_Left  = MinNode->BST_Right;
      DelNode = MinNode;                // hủy nút thế mạng
   }
   delete DelNode;
   return 1;
}
```

- Tham số `BST_Type &Root` là **chính liên kết của nút cha** ⇒ `Root = Root->BST_Right` là "nối tắt" cha với con.
- TH1 (lá) và TH2 (một con) xử lý chung bởi hai nhánh `if` đầu.
- TH3: **chép** Key của thế mạng lên Root, rồi xóa nút thế mạng (không có con trái ⇒ nối MinP với con phải của nó).
- Độ phức tạp: đi xuống tìm X rồi tìm thế mạng ⇒ tổng ≤ h ⇒ **O(h)**.

2.5 Minh họa hủy nút gốc có hai con

Ví dụ 11.6 – Tiếp tục hủy gốc 44 (hai con)

- Cây phải của 44 có gốc 88; đi trái đến hết: 88 → 59 → **55** (không có con trái) ⇒ thế mạng = 55, MinP = 59.
- Chép 55 vào nút gốc; nối 59.Left = 55.Right (= NULL); hủy nút 55 cũ.



- LNR: 15 23 37 55 59 71 88 108 – vẫn **tăng dần** ✓.

Các trường hợp đặc biệt

- **MinP == Root:** cây con phải không có nút trái (vd hủy 88 khi 88 không có 59) ⇒ thế mạng là chính con phải, nối Root->BST_Right.
- Nút **lá** rơi vào nhánh `BST_Left == NULL` ⇒ Root = BST_Right = NULL ✓.

- **Toàn bộ quá trình** trên cây B (ví dụ 11.5 – 11.6):

Hủy	Trường hợp	Thao tác	NLR sau khi hủy
40	Lá	37.Right = NULL	44 18 13 15 37 23 88 59 55 71 108
13	Một con (15)	18.Left = 15	44 18 15 37 23 88 59 55 71 108
18	Hai con	Thế mạng 23; 37.Left = NULL	44 23 15 37 88 59 55 71 108
44	Hai con	Thế mạng 55; 59.Left = NULL	55 23 15 37 88 59 71 108

- Sau mỗi lần hủy, kiểm tra lại: LNR vẫn **tăng dần**.

2.6 Thêm nút không đệ quy và tạo cây

```
int BST_Add_NR(BST_Type &Root, T NewData)
{  BST_Type Parent = NULL, p = Root;
   while (p != NULL)                    // tìm vị trí
   {  if (NewData == p->Key) return 0;  // đã có
      Parent = p;
      p = (NewData < p->Key) ? p->BST_Left
                             : p->BST_Right;
   }
   BST_Type NewNode = BST_Create_Node(NewData);
   if (NewNode == NULL) return 0;
   if (Parent == NULL) Root = NewNode;  // cây rỗng
   else if (NewData < Parent->Key)
      Parent->BST_Left = NewNode;
   else
      Parent->BST_Right = NewNode;
   return 1;
}
void BST_Create(BST_Type &Root, T M[], int N)
{  Root = NULL;
   for (int i = 0; i < N; i++)
      BST_Add_NR(Root, M[i]);
}
```

- Bản không đệ quy phải giữ con trỏ **Parent** (nút cha) để gắn nút mới – vì không còn "tham chiếu tới liên kết của cha" như bản đệ quy.

Ví dụ 11.4a – Thêm 50 vào cây B

p	So sánh	Parent
44	50 > 44 ⇒ phải	44
88	50 < 88 ⇒ trái	88
59	50 < 59 ⇒ trái	59
55	50 < 55 ⇒ trái	55
NULL	dừng	**55**

⇒ 55.Left = nút 50 (lá mới ở mức 5) ⇒ chiều cao cây tăng từ 4 lên **5**.

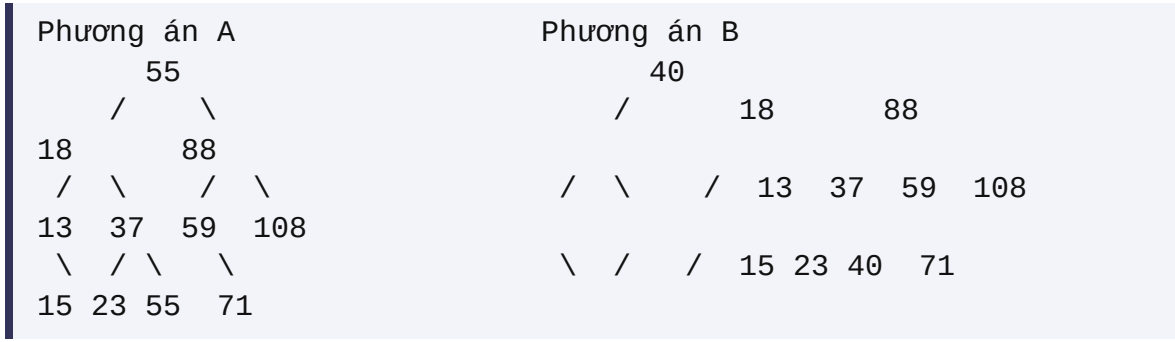
- **Tạo BST cân bằng từ mảng đã sắp:** chọn phần tử **giữa** làm gốc, đệ quy với hai nửa ⇒ chiều cao $\lceil \log_2(N + 1) \rceil$ (thay vì N nếu thêm tuần tự).

2.7 Hai phương án chọn phần tử thế mạng

- **Phương án A – trái nhất của cây con phải** (successor – phần tử **kế sau** trong LNR).
- **Phương án B – phải nhất của cây con trái** (predecessor – phần tử **kế trước** trong LNR).
- Cả hai đều cho BST hợp lệ; chọn một phương án và dùng thống nhất. Một số cài đặt **luân phiên** A, B để cây ít lệch.

Ví dụ 11.6a – Hủy 44 trên cây B ban đầu

- **A:** thế mạng = trái nhất cây phải = **55** ⇒ gốc mới 55; 59.Left = NULL.
- **B:** thế mạng = phải nhất cây trái = **40** ⇒ gốc mới 40; 37.Right = NULL.



- **Vì sao thế mạng có tối đa 1 con?** Trái nhất ⇒ không có con trái; phải nhất ⇒ không có con phải.
- Cài đặt phương án B: thay `BST_Right` ↔ `BST_Left` trong đoạn tìm thế mạng.

```
// Nút nhỏ nhất / lớn nhất: O(h)
BST_Type BST_Min(BST_Type Root)
{  if (Root == NULL) return NULL;
   while (Root->BST_Left != NULL) Root = Root->BST_Left;
   return Root;
}
BST_Type BST_Max(BST_Type Root)
{  if (Root == NULL) return NULL;
   while (Root->BST_Right != NULL) Root = Root->BST_Right;
   return Root;
}
// Đếm số nút có Key trong đoạn [a, b]
int BST_CountRange(BST_Type Root, T a, T b)
{  if (Root == NULL) return 0;
   if (Root->Key < a) return BST_CountRange(Root->BST_Right,
a, b);
   if (Root->Key > b) return BST_CountRange(Root->BST_Left,
a, b);
   return 1 + BST_CountRange(Root->BST_Left, a, b)
            + BST_CountRange(Root->BST_Right, a, b);
}
```

Ví dụ 11.7 – Trên cây B

- BST_Min = **13**, BST_Max = **108**.
- Đếm Key trong [20, 60]: 23, 37, 40, 44, 55, 59 ⇒ **6**. Tại nút 13 < 20 chỉ xét cây phải; tại 88 > 60 chỉ xét cây trái ⇒ **cắt tỉa** nhiều nhánh.

- **Kiểm tra một cây nhị phân có phải BST:** duyệt LNR và kiểm tra dãy **tăng nghiêm ngặt**; hoặc đệ quy truyền **khoảng hợp lệ** (min, max) xuống các cây con.

```
int LaBST(BST_Type p, long Min, long Max)
{  if (p == NULL) return 1;
   if (p->Key <= Min || p->Key >= Max) return 0;
   return LaBST(p->BST_Left, Min, p->Key)
       && LaBST(p->BST_Right, p->Key, Max);
}
```

- Ví dụ 11.2: tại 35, khoảng hợp lệ là (20, 30) ⇒ 35 ≥ 30 ⇒ **không phải BST**.

3.1 Độ phức tạp theo chiều cao cây

- Tìm kiếm, thêm, hủy đều đi theo **một đường** từ gốc xuống ⇒ **O(h)**, h là chiều cao cây.

Hình dạng cây	Chiều cao h	Tìm / thêm / hủy	N = 10^6
Cân bằng (gần hoàn chỉnh)	≈ $\log_2 N$	O(log N)	≈ 20 bước
Ngẫu nhiên (thêm theo thứ tự ngẫu nhiên)	trung bình ≈ 1,39·$\log_2 N$	O(log N)	≈ 28 bước
Suy biến (thêm theo thứ tự tăng/giảm)	N	**O(N)**	10^6 bước – như DSLK

Ví dụ 11.8 – Thêm 10, 20, 30, 40, 50 theo thứ tự tăng

```
10
  \
   20
     \
      30
        \
         40
           \
            50
```

- Cây **suy biến** thành danh sách liên kết; tìm 50 cần 5 phép so sánh = N.
- Cùng dữ liệu, thêm theo thứ tự 30, 20, 40, 10, 50 ⇒ cây chiều cao 3.

Ví dụ 11.9 – Ảnh hưởng thực tế

Nhập từ điển 100.000 từ **đã sắp xếp theo vần A–Z** vào BST ⇒ cây suy biến: tạo cây tốn ≈ $N^2/2$ = **$5 \cdot 10^9$** phép so sánh, mỗi lần tra từ trung bình 50.000 bước. Nếu nhập ngẫu nhiên: ≈ 23 bước mỗi lần tra.

- **Giải pháp:**
  - Xáo trộn dữ liệu trước khi thêm (không luôn làm được).
  - Dùng **cây cân bằng** tự điều chỉnh sau mỗi lần thêm/hủy: **AVL** (buổi 12), đỏ – đen, B-cây.

3.2 So sánh BST với các cấu trúc đã học

Thao tác	Mảng không thứ tự	Mảng có thứ tự	DSLK	BST (trung bình)	BST (suy biến)	AVL
Tìm kiếm	O(N)	O(log N)	O(N)	O(log N)	O(N)	O(log N)
Thêm	O(1)	O(N)	O(1)	O(log N)	O(N)	O(log N)
Hủy	O(N)	O(N)	O(N)*	O(log N)	O(N)	O(log N)
Duyệt theo thứ tự	O(N log N) (phải sắp)	O(N)	O(N log N)	O(N) – LNR	O(N)	O(N)
Tìm min / max	O(N)	O(1)	O(N)	O(log N)	O(N)	O(log N)

- (*) O(1) nếu đã biết vị trí nút (DSLK kép).

Ví dụ 11.10 – Lựa chọn cấu trúc

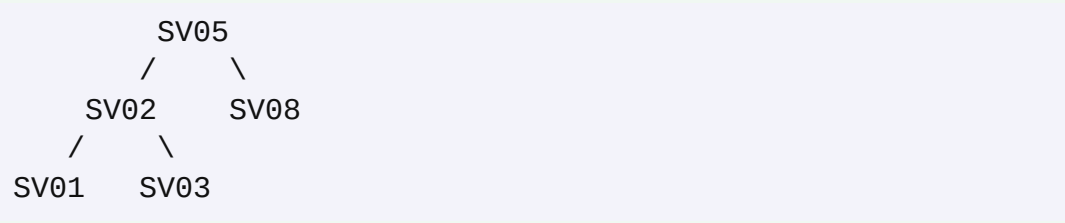
- Danh bạ 10.000 số, tra cứu nhiều, thỉnh thoảng thêm ⇒ **mảng có thứ tự** (đơn giản) hoặc **BST cân bằng**.
- Từ điển liên tục thêm/xóa/tra, cần in theo vần ⇒ **BST cân bằng** (AVL).
- Hàng chờ xử lý theo thứ tự đến ⇒ **Queue** (không cần BST).
- Nhật ký chỉ thêm vào cuối, đọc tuần tự ⇒ **DSLK** có Last.

3.3 BST với khóa là chuỗi và dữ liệu kèm theo

- Thực tế, khóa thường là **chuỗi** (từ, họ tên, mã SV) và mỗi nút mang thêm **dữ liệu**. Thay phép so sánh `<`, `==` bằng **strcmp**.

```
typedef struct SV_Node {
   char MaSV[11];        // khóa
   char HoTen[31];
   float DTB;            // dữ liệu kèm theo
   SV_Node *Left, *Right;
} SV_Node;
SV_Node *TimSV(SV_Node *Root, const char *Ma)
{  while (Root != NULL)
   {  int c = strcmp(Ma, Root->MaSV);
      if (c == 0) return Root;
      Root = (c < 0) ? Root->Left : Root->Right;
   }
   return NULL;
}
```

Ví dụ 11.11 – Thêm SV05, SV02, SV08, SV01, SV03




```
int ThemSV(SV_Node *&Root, const char *Ma,
           const char *Ten, float Dtb)
{  if (Root == NULL)
   {  Root = new SV_Node;
      strcpy(Root->MaSV, Ma);
      strcpy(Root->HoTen, Ten);
      Root->DTB = Dtb;
      Root->Left = Root->Right = NULL;
      return 1;
   }
   int c = strcmp(Ma, Root->MaSV);
   if (c == 0) return 0;
   if (c < 0) return ThemSV(Root->Left, Ma, Ten, Dtb);
   return ThemSV(Root->Right, Ma, Ten, Dtb);
}
```

- `SV_Node *&Root`: tham chiếu tới con trỏ (giống `BST_Type &Root`).
- strcmp so sánh theo **từ điển**: "SV10" < "SV9" ⇒ nên dùng mã **độ dài cố định** (SV09, SV10).

4.1 Bài thực hành buổi 11 – Từ điển Anh – Việt bằng BST

Yêu cầu

```
typedef struct Word_Node {
   char Anh[31];
   char Viet[101];
   Word_Node *Left, *Right;
} Word_Node;
typedef Word_Node *Dict;
```

- 1) Đọc từ điển từ tập tin văn bản (mỗi dòng: `tu_tieng_anh:nghia_tieng_viet`), thêm vào BST theo khóa Anh (so sánh bằng **strcmp**).
- 2) Tra từ: nhập từ tiếng Anh ⇒ in nghĩa; đếm số phép so sánh chuỗi đã dùng.
- 3) Thêm từ mới, sửa nghĩa, **xóa** từ (đủ 3 trường hợp hủy nút).
- 4) In toàn bộ từ điển theo vần A–Z (LNR); in các từ bắt đầu bằng một tiền tố (gợi ý: dùng khoảng [tiền tố, tiền tố + "~"] để cắt tỉa như BST_CountRange).
- 5) Ghi từ điển ra tập tin theo **NLR** – khi đọc lại sẽ được **đúng cây cũ** (giải thích vì sao). Thử ghi theo LNR rồi đọc lại: chiều cao cây thay đổi thế nào?
- 6) In **chiều cao cây**, số nút; so sánh khi nhập tập tin đã sắp theo vần và khi đã xáo trộn.

Gợi ý

Câu 5: ghi theo LNR tạo ra tập tin **đã có thứ tự** ⇒ đọc lại sinh ra cây **suy biến** (chiều cao = N) – minh họa trực tiếp cho mục 3.1 và động lực học cây AVL ở buổi 12.

4.2 Câu hỏi và bài tập

Câu hỏi

- 1) Định nghĩa BST. Vì sao duyệt LNR trên BST cho dãy tăng dần?
- 2) Khi hủy nút có hai con, phần tử thế mạng là gì? Vì sao phần tử thế mạng có tối đa một con?
- 3) Khi nào BST bị suy biến? Hậu quả và cách khắc phục?

Bài tập (có đáp số)

- 1) Vẽ BST khi thêm lần lượt **50, 30, 70, 20, 40, 60, 80, 35, 45, 65**; cho kết quả NLR, chiều cao. → NLR: **50 30 20 40 35 45 70 60 65 80**; chiều cao **4**.
- 2) Trên cây câu 1, hủy lần lượt 20, 30, 50 (thế mạng = trái nhất cây phải); vẽ cây sau mỗi bước. → Hủy 20 (lá); hủy 30 (một con 40) ⇒ 50.Left = 40; hủy 50 ⇒ thế mạng **60**, 70.Left = 65 ⇒ NLR cuối: **60 40 35 45 70 65 80**.
- 3) Cho NLR của một BST: 40 25 10 30 60 50 70. Vẽ cây (gợi ý: NLR của BST đủ để khôi phục cây vì LNR chính là dãy sắp xếp).
- 4) Viết hàm tìm phần tử **nhỏ thứ k** trong BST; hàm in các nút có Key trong [a, b] theo thứ tự tăng.